



Chapter 1: Introduction

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Database Applications Examples

- **Enterprise Information**
 - Sales: customers, products, purchases
 - Accounting: payments, receipts, assets
 - Human Resources: Information about employees, salaries, payroll taxes.
- **Manufacturing**: management of production, inventory, orders, supply chain.
- **Banking and finance**
 - customer information, accounts, loans, and banking transactions.
 - Credit card transactions
 - Finance: sales and purchases of financial instruments (e.g., stocks and bonds; storing real-time market data
- **Universities**: registration, grades



Database Applications Examples (Cont.)

- **Airlines:** reservations, schedules
- **Telecommunication:** records of calls, texts, and data usage, generating monthly bills, maintaining balances on prepaid calling cards
- **Web-based services**
 - Online retailers: order tracking, customized recommendations
 - Online advertisements
- **Document databases**
- **Navigation systems:** For maintaining the locations of various places of interest along with the exact routes of roads, train systems, buses, etc.



Purpose of Database Systems

In the early days, database applications were built directly on top of file systems, which leads to:

- **Data redundancy and inconsistency:** data is stored in multiple file formats resulting in duplication of information in different files
- **Difficulty in accessing data**
 - Need to write a new program to carry out each new task
- **Data isolation**
 - Multiple files and formats
- **Integrity problems**
 - Integrity constraints (e.g., account balance > 0) become “buried” in program code rather than being stated explicitly
 - Hard to add new constraints or change existing ones



Purpose of Database Systems (Cont.)

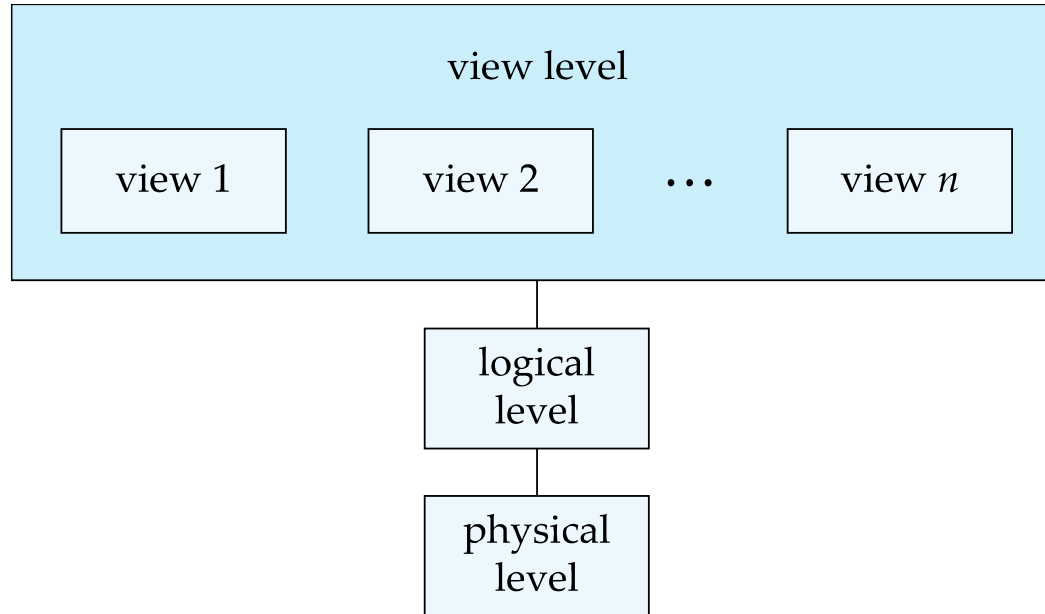
- **Atomicity of updates**
 - Failures may leave database in an inconsistent state with partial updates carried out
 - Example: Transfer of funds from one account to another should either complete or not happen at all
- **Concurrent access by multiple users**
 - Concurrent access needed for performance
 - Uncontrolled concurrent accesses can lead to inconsistencies
 - Ex: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- **Security problems**
 - Hard to provide user access to some, but not all, data

Database systems offer solutions to all the above problems



View of Data

An architecture for a database system





Instances and Schemas

- Similar to types and variables in programming languages
- **Schema** – the overall design of the database
 - Corresponds to the variable declarations (along with associated type definitions) in a program.
- **Instance** – the actual content of the database at a particular point in time
 - Analogous to the value of a variable



Instances and Schemas

Types of Schema

- **Logical Schema** – the overall logical structure of the database
 - Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them
 - Analogous to type information of a variable in a program
- **Physical schema** – the overall physical structure of the database
- **Sub schema** – A database may also have several schemas at the view level, sometimes



Related Terms

(Physical Data Independence)

- **Physical Data Independence** – the ability to modify the physical schema without changing the logical schema
 - Applications depend on the logical schema
 - In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.



Data Models

- A **collection of tools** for describing
 - Data
 - Data relationships
 - Data semantics
 - Data constraints

Data Models can be classified into 4 categories:

- Relational model
- Entity-Relationship data model (mainly for database design)
- Object-based data models (Object-oriented and Object-relational)
- Semi-structured data model (XML)
- Other older models:
 - Network model
 - Hierarchical model



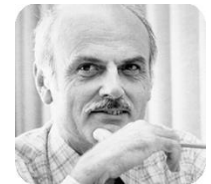
Data Model-1: Relational Model

- All the data is stored in various tables.
- Tables are known as ***relations***
- Example of tabular data in the relational model

Columns

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows



Ted Codd
Turing Award 1981

(a) The *instructor* table



A Sample Relational Database

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table



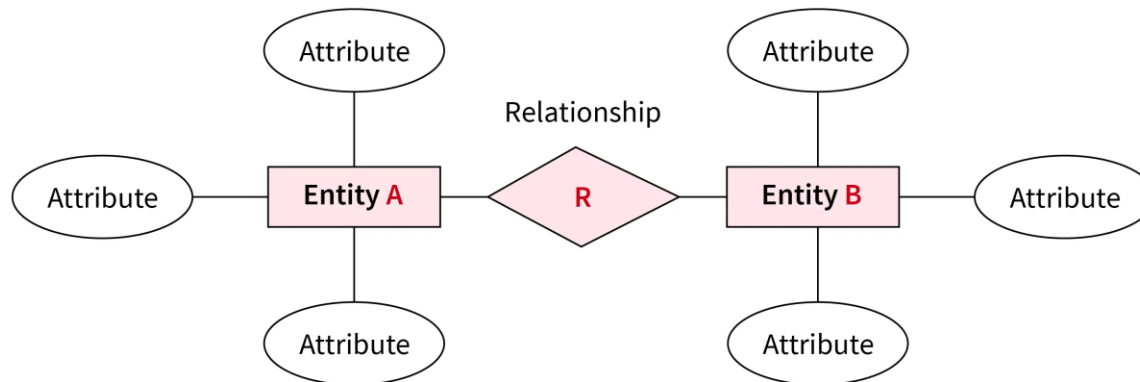
Relational Database

- A **relational database** is based on the relational model and uses a collection of tables to represent both data and the relationships among those data.
- It also includes a DML and DDL.
- Most commercial relational database systems employ the **SQL language**,



Data Model-2: Entity Relationship Model

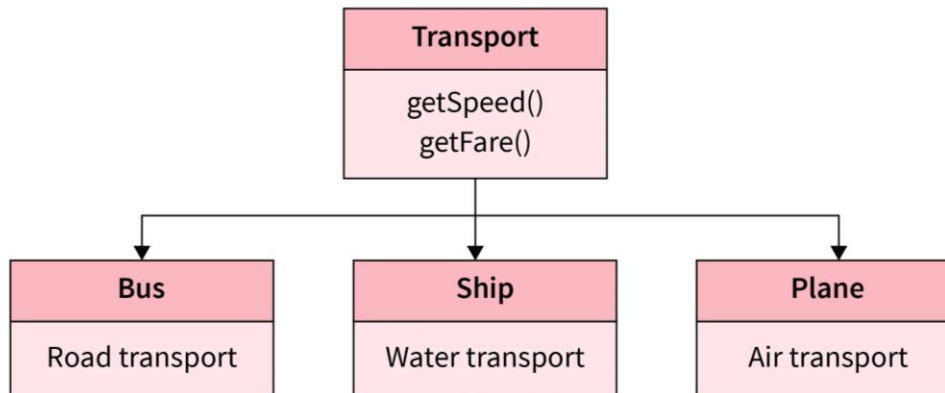
- A collection of basic objects, called entities, and relationships among these objects.
- An entity is a “thing” or “object” in the real world that is distinguishable from other objects





Data Model-3: Object-based data Model

- An object data model is a data model based on object-oriented programming, associating methods (procedures) with objects that can benefit from class hierarchies.
- Combines features of the object-oriented data model and relational data model

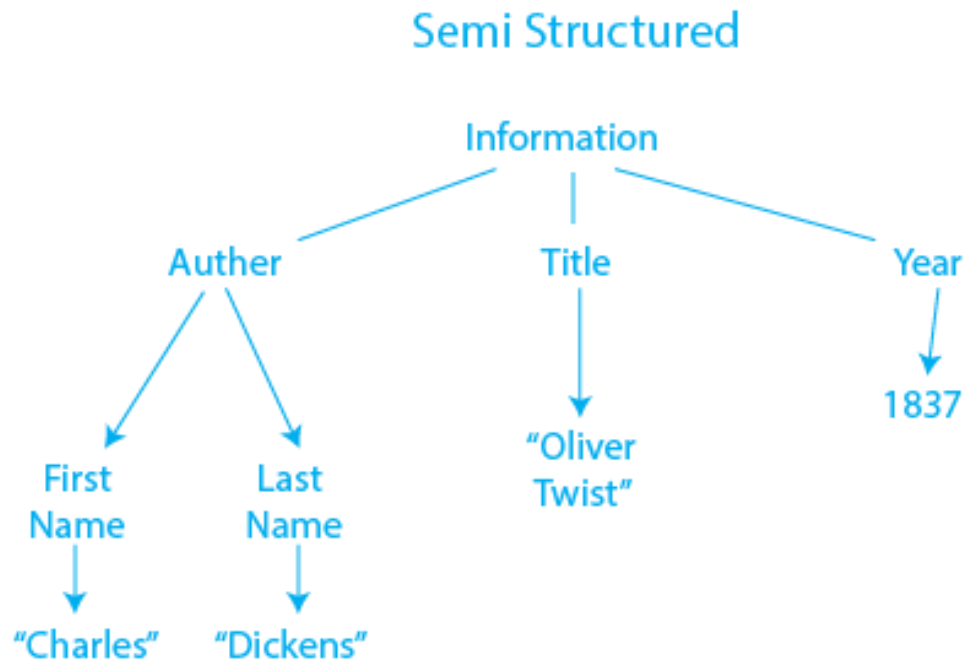


- Transport, Bus, Ship, and Plane are objects.
- Bus has Road Transport as the attribute.
- Ship has Water Transport as the attribute.
- Plane has Air Transport as the attribute.
- The Transport object` is the base object and the Bus, Ship, and Plane objects derive from it.



Data Model- 4: Semi-Structured data Model

- The semi-structured data model permits the **specification of data where individual data items of the same type** may have different sets of attributes.
- The ***Extensible Markup Language (XML)*** is widely used to represent semi-structured data.





Database Languages

- ❖ the *data-definition* and *data-manipulation languages* are not two separate languages;
- ❖ instead they simply form parts of a single database language, such as the widely used **SQL** language.

- **Data Definition Language**
- **Data Manipulation Language**



Data Definition Language (DDL)

- Specification notation for defining the **database schema**

Example: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

- DDL compiler generates a set of table templates stored in a **data dictionary**
- The DDL, just like any other programming language, gets as **input** some instructions (**statements**) and **generates some output**.
- The output of the DDL is placed in the **data dictionary**, which contains **metadata**—that is, data about data.



Data Definition Language (DDL)

- ❑ The data values stored in the database must satisfy certain **consistency constraints**.
 - The DDL provides facilities to **specify such constraints**.
 - The database system checks these constraints every time the database is updated.
 - Database constraints
 - **Domain constraints**
 - A domain of possible values must be associated with every attribute (for example, integer types, character types, date/time types).
 - **Referential Integrity**
 - There are cases where we wish to ensure that a value that appears in one relation for a given set of attributes also appears in a certain set of attributes in another relation (referential integrity).
- Example:** the **dept_name** value in a **course** record must appear in the **dept_name** attribute of some record of the **department** relation. Database modifications can cause violations of referential integrity.



Data Definition Language (DDL)

- Database constraints
 - **Assertions**
 - An assertion is any condition that the database must always satisfy.
 - Domain constraints and referential-integrity constraints are special forms of assertions..
 - **Example:** *“Every department must have at least five courses offered every semester”* must be expressed as an assertion. When an assertion is created, the system tests it for validity.
 - **Authorization**
 - Differentiations in the type of access they are permitted on various data values in the database is called “authorization”
 - **read authorization:** which allows reading, but not modification, of data
 - **insert authorization:** which allows insertion of new data, but not modification of existing data
 - **update authorization,** which allows modification, but not deletion, of data;
 - **delete authorization:** which allows deletion of data.



Special Type of DDL

- **Data storage and definition language:**
 - the ***storage structure and access methods*** used by the database system are specified by a set of statements in a special type of DDL called a data storage and definition language.
 - statements define the implementation details of the database schemas, which are usually hidden from the users.



Data Definition Language (DDL)

- SQL provides a rich DDL that allows one to define tables, integrity constraints, assertions, etc.
- For instance, the following SQL DDL statement defines the department table:

```
create table department  
  (dept_name    char (20),  
   building     char (15),  
   budget       numeric (12,2));
```



Data Manipulation Language (DML)

- Language for accessing and updating the data organized by the appropriate data model
 - DML also known as query language
- There are basically two types of data-manipulation language
 - **Procedural DML** -- require a user to specify what data are needed and how to get those data.
 - **Declarative DML** -- require a user to specify what data are needed without specifying how to get those data.
- Declarative DMLs are usually easier to learn and use than are procedural DMLs.
- Declarative DMLs are also referred to as non-procedural DMLs
- The portion of a DML that involves information retrieval is called a **query** language.



Data Manipulation Language (DML)

- The SQL query language is nonprocedural
- SQL query that finds the names of all instructors in the History department

```
select instructor.name  
from instructor  
where instructor.dept_name = 'History';
```

- Query finds the instructor ID and department name of all instructors associated with a department with budget of greater than \$95,000.

```
select instructor.ID, department.dept_name  
from instructor, department  
where instructor.dept_name= department.dept_name and  
      department.budget > 95000;
```




SQL Query Language

- SQL query language is nonprocedural. A query takes as input several tables (possibly only one) and always returns a single table.

- Example to find all instructors in Comp. Sci. dept

```
select name  
from instructor  
where dept_name = 'Comp. Sci.'
```

- SQL is **NOT** a Turing machine equivalent language
- To be able to compute complex functions SQL is usually embedded in some higher-level language
- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database



Database Access from Application Program

- Non-procedural query languages such as SQL are not as powerful as a universal Turing machine.
- SQL does not support actions such as input from users, output to displays, or communication over the network.
- Such computations and actions must be written in a **host language**, such as C/C++, Java or Python, with embedded SQL queries that access the data in the database.
- **Application programs** -- are programs that are used to interact with the database in this fashion.



Database Design

The process of designing the general structure of the database:

- Logical Design – Deciding on the database schema. Database design requires that we find a “good” collection of relation schemas.
 - Business decision – What attributes should we record in the database?
 - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- Physical Design – Deciding on the physical layout of the database



Database Engine

- A database system is partitioned into modules that deal with each of the responsibilities of the overall system.
- The functional components of a database system can be divided into
 - The storage manager,
 - The query processor component,
 - The transaction management component.



Database Sub-Systems

- A database system has **several subsystems**.
 - The **storage manager** subsystem provides the interface between the lowlevel data stored in the database and the application programs and queries submitted to the system.
 - The **query processor** subsystem compiles and executes DDL and DML statement



Storage Manager

- A program module that **provides the interface between the low-level data stored in the database and the application programs and queries** submitted to the system.
- The **raw data** are stored on the disk using the file system provided by the operating system.
- The storage manager **translates the various DML statements into low-level file-system** commands.
- **Why storage manager is important?**
 - As databases typically require a large amount of storage space
 - Corporate databases range in size from hundreds of gigabytes to, for the largest databases, terabytes of data.



Storage Manager

- The **storage manager is responsible** to the following tasks:
 - Interaction with the OS file manager
 - Efficient storing, retrieving and updating of data

- The **storage manager components** include:
 - Authorization and integrity manager
 - Transaction manager
 - File manager
 - Buffer manager



Storage Manager (Cont.)

- The storage manager implements **several data structures** as part of the physical system implementation:
 - **Data files** -- store the database itself
 - **Data dictionary** -- stores metadata about the structure of the database, in particular the schema of the database.
 - **Indices** -- can provide fast access to data items. A database index provides pointers to those data items that hold a particular value.



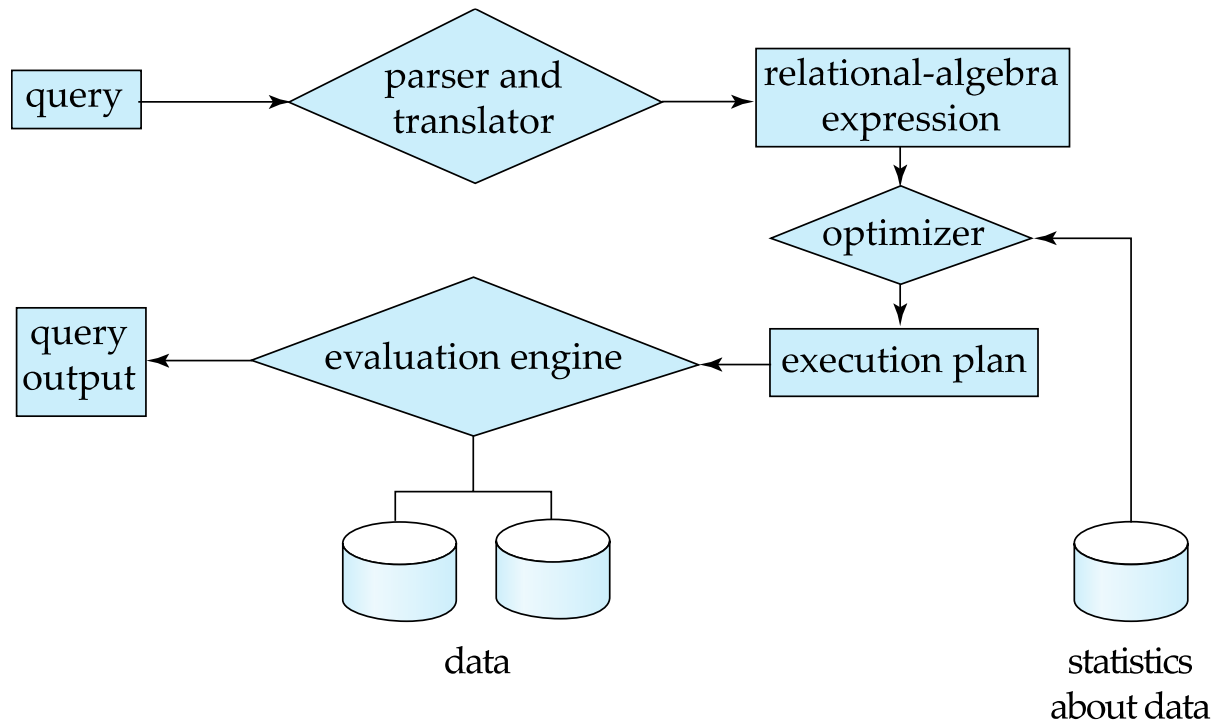
Query Processor

- Why query processor important?
 - it helps the database system to simplify and facilitate access to data.
 - allows database users to obtain good performance
- The **query processor components** include:
 - **DDL interpreter** -- interprets DDL statements and records the definitions in the data dictionary.
 - **DML compiler** -- translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.
 - The DML compiler performs **query optimization**; that is, it picks the lowest cost evaluation plan from among the various alternatives.
 - **Query evaluation engine** -- executes low-level instructions generated by the DML compiler.



Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation





Transaction Management

- A **transaction** is a collection of operations that performs a single logical function in a database application
- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.
- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.

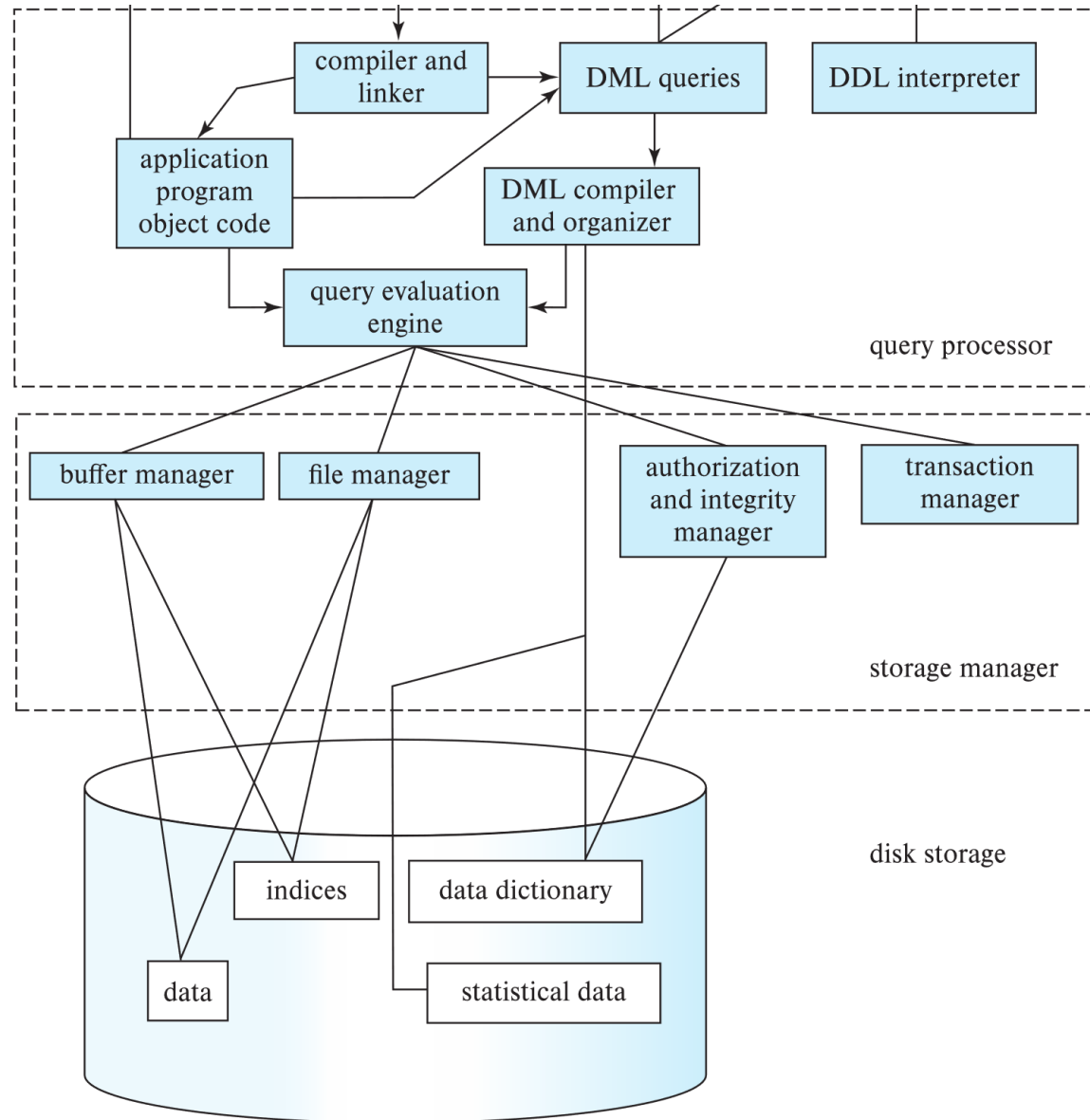


Database Architecture

- Centralized databases
 - One to a few cores, shared memory
- Client-server,
 - One server machine executes work on behalf of multiple client machines.
- Parallel databases
 - Many core shared memory
 - Shared disk
 - Shared nothing
- Distributed databases
 - Geographical distribution
 - Schema/data heterogeneity



Database Architecture (Centralized/Shared-Memory)





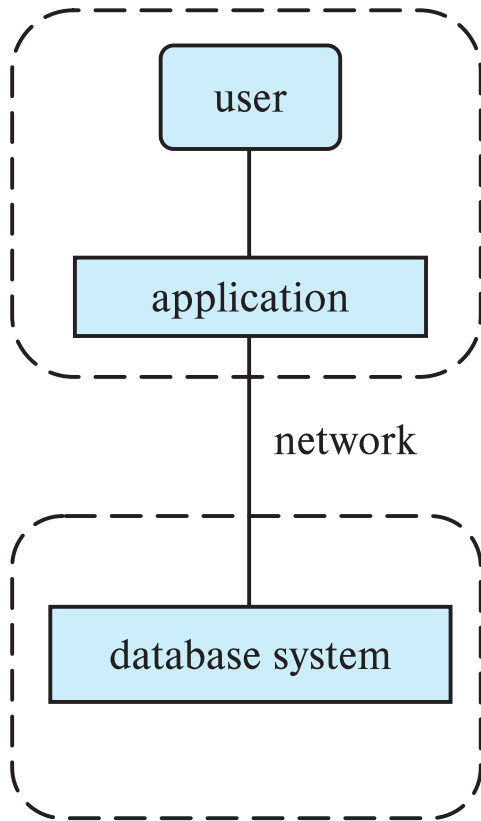
Database Applications

Database applications are usually partitioned into two or three parts

- **Two-tier architecture** -- the application resides at the client machine, where it invokes database system functionality at the server machine
- **Three-tier architecture** -- the client machine acts as a front end and does not contain any direct database calls.
 - The client end communicates with an application server, usually through a forms interface.
 - The application server in turn communicates with a database system to access data.



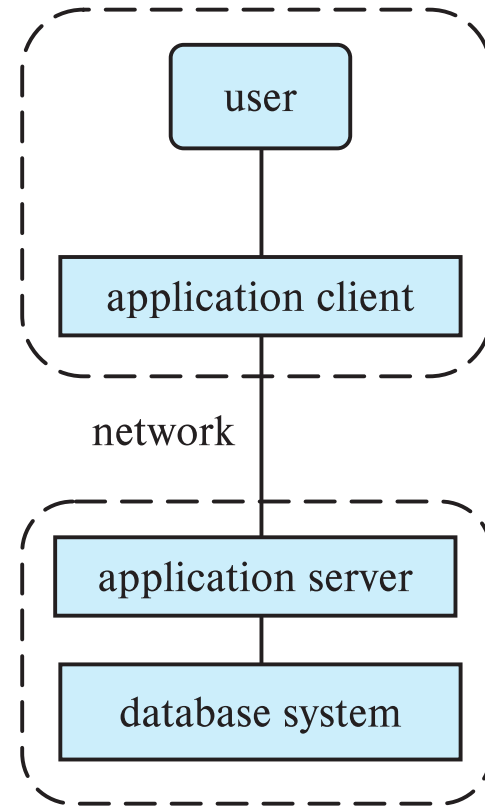
Two-tier and three-tier architectures



(a) Two-tier architecture

client

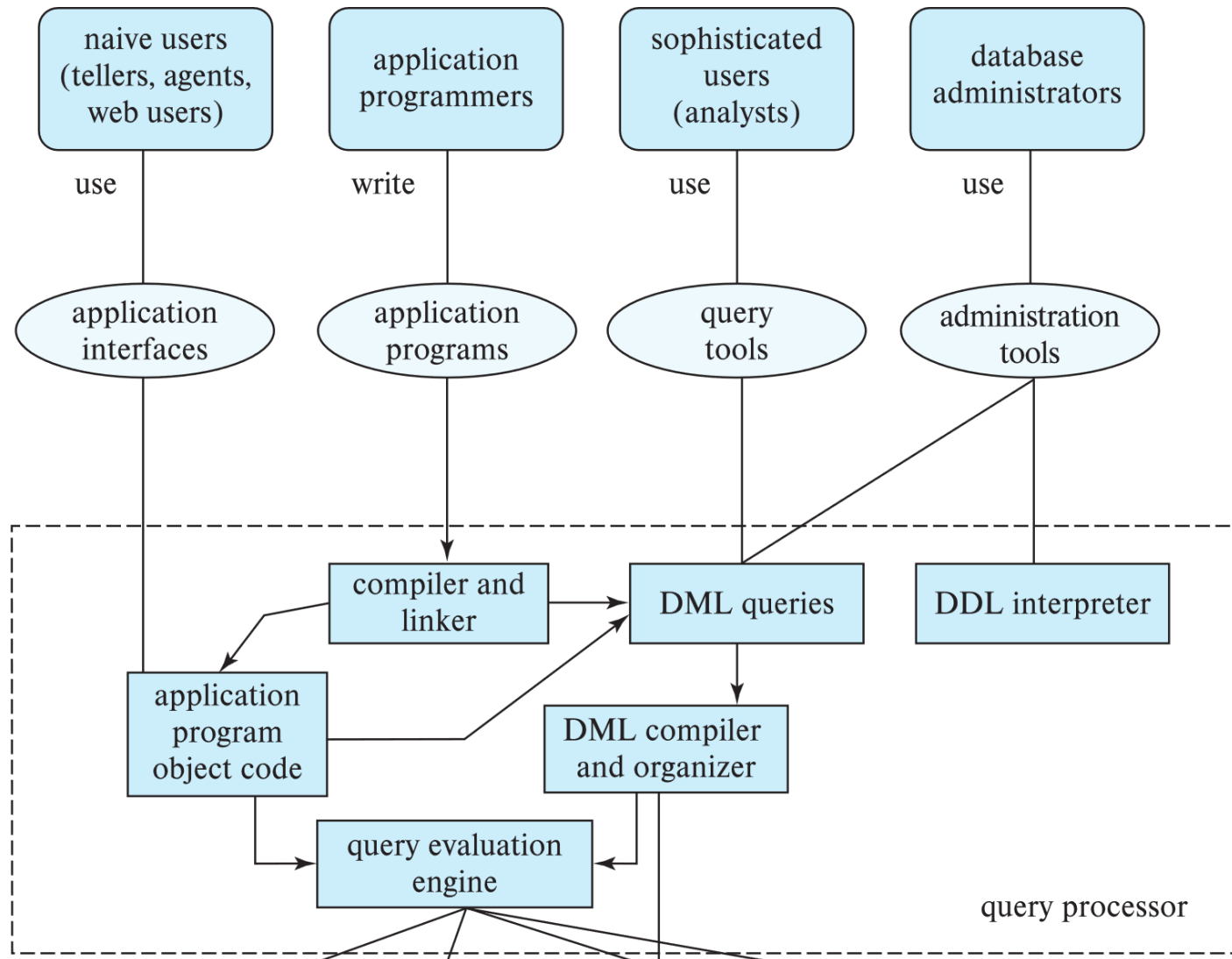
server



(b) Three-tier architecture



Database Users





Database Users

- **Naive users:** unsophisticated users who interact with the system by invoking one of the application programs that have been written previously.
- For example, a clerk in the university who needs to add a new instructor to department A invokes a program called new hire. This program asks the clerk for the name of the new instructor, her new ID, the name of the department (that is, A), and the salary.
- **Application programmers** are computer professionals who write application programs.
- Application programmers can choose from many tools to develop user interfaces.
- Rapid application development (RAD) tools are tools that enable an application programmer to construct forms and reports with minimal programming effort.



Database Users

- **Sophisticated users** interact with the system without writing programs.
- Instead, they form their requests either using a database query language or by using tools such as data analysis software.
- **Specialized users** are sophisticated users who write specialized database applications that do not fit into the traditional data-processing framework.
- Among these applications are computer-aided design systems, knowledgebase
- and expert systems, systems that store data with complex data types



Database Administrator

A person who has central control over the system is called a **database administrator (DBA)**. Functions of a DBA include:

- **Schema definition:** The DBA creates the original database schema by executing a set of data definition statements in the DDL.
- **Storage structure and access-method definition**
- **Schema and physical-organization modification:** The DBA carries out changes to the schema and physical organization to reflect the changing needs of the organization, or to alter the physical organization to improve performance.
- **Granting of authorization for data access:** By granting different types of authorization, the database administrator can regulate which parts of the database various users can access.



Database Administrator

- **Routine maintenance:** Examples of the database administrator's routine maintenance activities are:
 - Periodically backing up the database
 - Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required
 - Monitoring jobs running on the database



History of Database Systems

- 1950s and early 1960s:
 - Data processing using magnetic tapes for storage
 - Tapes provided only sequential access
 - Punched cards for input
- Late 1960s and 1970s:
 - Hard disks allowed direct access to data
 - Network and hierarchical data models in widespread use
 - Ted Codd defines the relational data model
 - Would win the ACM Turing Award for this work
 - IBM Research begins System R prototype
 - UC Berkeley (Michael Stonebraker) begins Ingres prototype
 - Oracle releases first commercial relational database
 - High-performance (for the era) transaction processing



History of Database Systems (Cont.)

- 1980s:
 - Research relational prototypes evolve into commercial systems
 - SQL becomes industrial standard
 - Parallel and distributed database systems
 - Wisconsin, IBM, Teradata
 - Object-oriented database systems
- 1990s:
 - Large decision support and data-mining applications
 - Large multi-terabyte data warehouses
 - Emergence of Web commerce



History of Database Systems (Cont.)

- 2000s
 - Big data storage systems
 - Google BigTable, Yahoo PNuts, Amazon,
 - “NoSQL” systems.
 - Big data analysis: beyond SQL
 - Map reduce and friends
- 2010s
 - SQL reloaded
 - SQL front end to Map Reduce systems
 - Massively parallel database systems
 - Multi-core main-memory databases



End of Chapter 1